

Qualification details			
Training Package Code and Title:	ICT - Information and Communications Technology		
Qualification National Code and Title:	ICT50220 Diploma of Information Technology (Advanced programming)	State code	AC21
Assessment Title	AT2 – Practical		
Unit National Code & Title	ICTPRG549 Apply intermediate object-oriented language skills ICTPRG533 Debug and monitor applications		
Date Due	Refer to the DAP	Date Received	
Student Name		Student ID	
Student Declaration	I declare that the evidence submitted is my own work:		
Assessor Name			
Assessment Decision	☐ Satisfactory	☐ Not Yet Satisfactory	
Assessor Signature		Date	
Is student eligible for reassessment (Re-sit)?	☐ No	☐ Yes	Reassessment Date:
Feedback to student			
Via Blackboard (LMS) – Please check [Grade] section.			
Feedback from student			
Via Blackboard (LMS) – Please use [Comment] section during submission.			
Student signature		Date	

Assessment Instructions

TO THE ASSESSOR

Type of Assessment	The analysis, design, coding, testing and project documentation of a Java application as described on the following page.
Duration of Assessment	Week 13 ~ 18
Location of Assessment	In class
Conditions	Assessor to ensure that the noise levels, natural interactions and time variances are maintained as it would in the be in the programming industry. Learners are required to complete the required tasks and submit the required documentation electronically via Blackboard
Elements and Criteria	As detailed in the assessment plan You are required to make sure that all students meet the elements, performance criteria and oral communication items as outlined in the provided checklist.

TO THE STUDENT

Purpose of Assessment	You are required to show you have knowledge of the following: ICTPRG549 Apply intermediate object-oriented language skills - design and build at least one simple application program from a problem scenario and program specification. In the course of the above, the candidate must: - use different object-oriented programming language techniques - check code optimisation. ICTPRG533 Debug and monitor applications - implement changes to improve an application's performance on at least one application. In the course of the above, the candidate must: - collect application performance data - identify and analyse performance issues - use a logging framework to create an event log for an application - use tools and techniques to trace and debug an application. You are required to meet the elements, performance criteria and oral communication items as outlined in the provided checklist.
Allowable Materials	Blackboard (Topic by topic) will include the following: Weekly Readings, Class notes, and Weekly Activities.

Assessment Coversheet

AT3 – Practical (OOP 2)

Required Resources	Computer lab with MS-Windows PC / Programming tools: JDK, Kotlin Compiler, IntelliJ IDEA, Android Studio / Blackboard Account / Internet Access
Reasonable Adjustment	In some circumstances, adjustments to assessments may be made for you. If you require support for literacy and numeracy issues; support for hearing, sight or mobility issues; change to assessment times/venues; use of special or adaptive technology; considerations relating to age, gender and cultural beliefs; format of assessment materials; or presence of a scribe you need to inform your lecturer.
Assessment Submission	All questions and activities must be attempted. Use of research tools and peers in formulating answers are acceptable – but work submitted must be your own work. Final work files and documentation are to be uploaded to the appropriate area in the Blackboard course created for this unit. If you are marked as NYS (Not Yet Satisfactory) on your first attempt, you will be provided with another opportunity to re-attempt the assessment.
Project contents	This assessment consists of the questions listed on the following pages:

Assessment Specification**NOTE:**

- The job role environment is that you are working as a junior programmer in a software development company. You will develop small-sized applications with Object Oriented Programming knowledge and skills.
- There are different types of questions
 - If the question requires performing/demonstrating some processes, screenshots are required
 - In class demonstrations may be required for further evidence
 - If Kotlin programming is required, include your source files (xxx.kt) and provide screenshots of program running

- Submit your source code with JAR demo application
 - Your source code must be in **src** folder
 - You can submit the **src** folder of your Kotlin project
 - Your programs will be tested with JetBrains IntelliJ IDEA using JDK
 - Provide JAR demo application with **.bat** file
 - This batch file runs your JAR application
 - Do NOT submit the whole IDE project folders
 - Refer to Kotlin/Java API specification for available classes and methods
 - If Android mobile programming is required, submit the whole project files and provide screenshots of program running

- Zip each project to a single file with the project name
 - Provide app APK file
 - Your APK must be signed and in release mode
 - If answering a question is required, write your answers in MS-WORD files
 - With all other types of questions, save your answers appropriately
 - All programming code must comply with the Kotlin and Android Code Conventions
 - IntelliJ IDEA and Android Studio provide code reformat option
- Zip all your files into a single .zip file before submit.
 - Save your zip file as the submit name, e.g. Practical.zip.
- To ensure authenticity, the students may be required to demonstrate in class for assessment
- **Cheating and plagiarism may result in unit re-enrol.**

Activity 1

- SMT Business needs a customer search program
- Develop a text-based program that provides sort and search options in Kotlin
 - Define Customer class:
 - The Customer class has 4 class variables: id, name, email and mobile
 - This class is in a sub-package named **objects**
 - Create 5 Customer objects in a collection data structure, e.g. *ArrayList*
 - The program shows the original (unsorted) customers' information
 - The program offers customers sort and search options
 - Provide sort options by name
 - Use any sortXX() methods of Kotlin's collection types
 - Provide search options by name, email or mobile
 - Full or part of details can be used for search
 - E.g. "Jone" can find "David Jones"
 - Multiple search results can be found
 - The program has an option to save the Customer objects in a binary file
 - Name the binary file as **customers.dat**
 - The program has an option to reads the customers object from the binary file and to display the customer contents on the screen
 - Design your own user interaction menu

- The UI of your program will be text-based
- Identify possible runtime errors, and improve program stability by using exception-handling techniques
- Save the main program file as **CustomerSortAndSearch**

Activity 2

- SMT Business needs a customer management Android mobile app with database
- Your app performs below:
 - Your application will access a database named **smtbiz**
 - This database has one table named **customer**
 - The customer table has 4 fields: Id, Name, Email and Mobile.

ID	Name	Email	Mobile
6 digits customer no. E.g. 123456 Primary Key	Full name. E.g. John Citizen		
 - Your app provides an option to reset the table with 5 records
 - This option will be used when your program runs for the first time or when it needs to reset the table with 5 records
 - Your app will provide options to perform CRUD database operations
 - Insert a new customer.
 - No validity check is required
 - E.g. wrong format of email or mobile can be accepted
 - Delete an existing customer by ID
 - Search for a customer by name
 - If the customer is not found, the app shows a friendly message
 - E.g. Android'Toast or Snackbar
 - Show all customers
 - Display all customers
 - Implement the Customer model class
 - Update a customer's info
 - A record found by ID is updated with the other details provided
 - Implement DB transaction integrity with the transaction methods of *SQLiteDatabase* class
 - Determine and document program requirements according to object-oriented programming specifications
 - Save and name your requirements document as "Programming Requirements."
 - Design your own GUI
 - Save your app project as **CustomerManagement**
- Demonstrate the use of Debugger and Profiler tools of Android Studio IDE
 - Show how you used the Debugger tool
 - Demonstrate variables examination and tracing running code
 - Demonstrate how the tools help process of development and increase the productivity
 - Write a test procedure and test document for test execution
 - Provide a test report according to test results. Use the test report template provided
 - Show how you used the Profiler tool
 - Analyse the performance of your app document any issues and improvement of performance
 - Provide an evidence of improvement, e.g different version

End of Assessment Tool